

CopernicusLAC Infrastructure Support

Interface Control Document

TERRADUE

Advancing Earth Science



UNIVERSIDAD
DE CHILE

T Systems



Title	Copernicus LAC Infra - D3 Interface Control Document
Prepared by	Terradue
Reference	T2-ESA-COPLACI-ID-24-030
Issue	1.3
Date	2025-07-27
Status	Issued
Document Type	Interface Design Document
Configuration	Configured document
ESA Contract No:	4000144570/24/I-KE

Change record

Date	Issue	Section	Page	Comment
2024-08-15	1.0	All	All	Initial issue
2024-12-03	1.1	All 2.3.1, 3	All 11-12, 14-17	• Typos • Include different levels of ingestion, metadata registration, dataset copying, and dataset calibration
2025-05-28	1.2	All 1.6	All 6	• Typos • Align applicable documents version
2025-07-27	1.3	6.1	31	• Clarified the use and scope of the signed URL data access interface in Section 6.1, including M2M usage, Web UI integration, and CWL workflow support.

Table of Content

1 Introduction	4
1.1 Purpose	4
1.2 Background	4
1.3 Scope	5
1.4 Audience	5
1.5 Document Structure	5
1.6 Applicable Documents	6
1.7 Reference Documents	6
1.8 Acronyms and Abbreviations	6
2 High-Level System Architecture	8
2.1 Overview	8
2.2 Key Components	9
2.3 Component Interactions	10
3 Data Ingestion	13
3.1 Overview	13
3.2 Interface Requirements	14
3.3 Data Formats and Protocols	15
3.4 Security and Compliance	15
3.5 Error Handling and Logging	16
3.6 Future Work	16
4 Data Discovery	17
4.1 Overview	17
4.2 Interface Description	17
4.3 Filtering	19
4.4 STAC Items	22
5 Data Visualisation	28
5.1 Overview	28
5.2 Visualisation Types	28
5.3 Overview Images	28
5.4 Web Map Tile Service (WMTS) Endpoints	29
6 Data Access Interfaces	31
6.1 Overview	31
6.2 Downloading Data with Signed URLs	31
6.3 Practical Example	34

1 Introduction

1.1 Purpose

The Interface Control Document (ICD) for the CopernicusLAC platform is designed to define and manage the interfaces between the platform and external systems, including data providers, third-party services, and external users. The primary goal of this document is to ensure that all interactions between the CopernicusLAC platform and external entities are clearly defined, secure, and compliant with relevant standards and regulations. This ICD provides a detailed description of the data exchange formats, protocols, and security measures employed by the platform to facilitate integration with external systems. By establishing these interfaces, the CopernicusLAC platform can effectively support data ingestion, discovery, visualisation, and access while maintaining high levels of performance, security, and scalability.

1.2 Background

The CopernicusLAC initiative is a collaborative effort between the European Space Agency (ESA), the European Commission's Directorate-General for International Partnerships (DG-INTPA), and the Government of Panama. The initiative aims to establish a Regional Copernicus Centre in Panama to facilitate the uptake and utilisation of Copernicus data in the LAC region. The primary focus of the Centre is to support disaster risk reduction and recovery efforts, leveraging Earth Observation (EO) data and geospatial technology to provide information services to local and regional stakeholders.

This initiative is part of the broader EU-LAC Digital Alliance, which promotes cooperation in digital and space technologies between the European Union (EU) and LAC countries. The CopernicusLAC Centre is envisioned as a pilot project under the EU's Global Gateway strategy, aiming to foster international collaboration, enhance data sharing, and support the development of local capacities in EO data processing and utilisation.

The CopernicusLAC Centre is a regional facility dedicated to the management, processing, and dissemination of EO data from the Copernicus Program, specifically tailored for the LAC region. The centre aims to provide comprehensive services for data hosting, processing, and access, supporting a wide range of applications, including environmental monitoring, disaster risk reduction, and sustainable development. The centre also focuses on enhancing local capacities by transferring knowledge and providing training to local operators.

The CopernicusLAC Centre Platform refers to the technical infrastructure and software environment that enables the functionalities of the CopernicusLAC Centre. This platform includes the hardware, software, middleware, and network components required to ingest, process, store, and distribute Copernicus EO data. The platform supports various workflows

and tools necessary for data management and analysis, ensuring seamless access and usability for different stakeholders. It is designed to be scalable, capable of handling large volumes of data and supporting a growing number of users and applications.

1.3 Scope

This document covers the interfaces between the CopernicusLAC platform and external systems, with a focus on four key functional areas: data ingestion, data discovery, data visualisation, and data access. The scope includes the specification of communication protocols, data formats, security measures, and compliance requirements that govern these interfaces. Additionally, the document outlines the testing, validation, and change management processes that ensure the reliability and integrity of the platform's external interactions.

1.4 Audience

The intended audience for this document includes system architects, software developers, system integrators, project managers, and other stakeholders involved in the development, deployment, and maintenance of the CopernicusLAC platform. It is also relevant for external partners and third-party service providers who need to understand how to integrate their systems with the CopernicusLAC platform, as well as for security and compliance officers responsible for ensuring that external interactions meet regulatory requirements.

1.5 Document Structure

This Service Specification document is divided into the following Chapters:

- Chapter 1 - Introduction (current): Provides an overview of the document, including its purpose, scope, intended audience, and structure..
- Section 2 - High-Level System Architecture: Describes the overall architecture of the CopernicusLAC platform, highlighting the key components and their roles within the system. This section serves as a foundation for understanding how the system components interact with external systems detailed in subsequent sections.
- Section 3 - Data Ingestion Interfaces: Details the interfaces related to the ingestion of data into the system, focusing on the integration with the Copernicus Data Space Ecosystem (CDSE). It includes specifications on data formats, protocols, and security measures.
- Section 4 - Data Discovery Interfaces: Focuses on the interfaces that enable data discovery within the system, specifically the STAC API. This section outlines the API endpoints, data structure, and security considerations.
- Section 5 - Data Visualisation Interfaces: Describes the interfaces used for data visualisation, including those related to Titiler and the Web Map Tile Service (WMTS).

It covers the technical requirements, data rendering processes, and API specifications.

- Section 6 - Data Access Interfaces: Addresses the interfaces involved in data access, particularly those using S3-compatible storage and Signed URLs. This section covers the data retrieval process, security measures, and performance considerations.

1.6 Applicable Documents

Ref.	Title and References	Issue
[SOW]	CopernicusLAC Infrastructure Support (EOP-EOP-SI-SOW-2023-15)	1.0
[CTR]	ESA Contract No. 4000140709/23/I-EF with Terradue for Platform Extensions with AI Capabilities	1.0
[D1]	T2-ESA-COPLAC-RS-24-028 D1 CopernicusLAC Centre Specification and Performance Requirements	1.4
[D2]	T2-ESA-COPLACI-DD-24-029 D2 System Architecture Description	1.3

1.7 Reference Documents

Ref.	Title and References	Issue
[PROP]	T2-ESA-AO1-11947-OF-23-062 Terradue Offer for CopLAC Infra (all Volumes)	1.0
[WMTS]	OGC WMTS Simple Profile	13-082r2
[OAF]	OGC API - Features	17-069r4
[OAP]	OGC API - Processes	18-062r2

1.8 Acronyms and Abbreviations

The following acronyms are used within this document:

API	Application Programming Interface
CDSE	Copernicus Data Space Ecosystem
COG	Cloud Optimised GeoTIFF
CWL	Common Workflow Language
DBaaS	Database as a Service

EO	Earth Observation
ESA	European Space Agency
EU	European Union
ETL	Extract, Transform, Load
GDPR	General Data Protection Regulation
HTTPS	HyperText Transfer Protocol Secure
ICD	Interface Control Document
LAC	Latin America and the Caribbean
M2M	Machine-To-Machine
OGC	Open Geospatial Consortium
RBAC	Role-Based Access Control
STAC	SpatioTemporal Asset Catalog
WMTS	Web Map Tile Service

2 High-Level System Architecture

The CopernicusLAC platform is designed as a modular, cloud-native system that supports the entire lifecycle of Earth Observation (EO) data, from acquisition and processing to discovery, access, and analysis. This section provides an overview of the platform's architecture, highlighting the key components and their roles within the system. Understanding this architecture is crucial for comprehending the interfaces detailed in subsequent sections.

This high-level system architecture aligns with the detailed descriptions provided in the D2 System Architecture Description document. The D2 document elaborates on the specific technologies, workflows, and interactions within each component, providing a deeper understanding of the platform's operation. For detailed diagrams, technical specifications, and component interactions, refer to the D2 System Architecture Description.

2.1 Overview

The CopernicusLAC platform is built on a distributed architecture that leverages modern cloud technologies to ensure scalability, flexibility, and resilience. The platform integrates multiple components, each responsible for specific functions within the EO data lifecycle. The architecture is designed to handle large volumes of data and complex workflows, ensuring that users can access and process EO data in a reliable and timely manner.

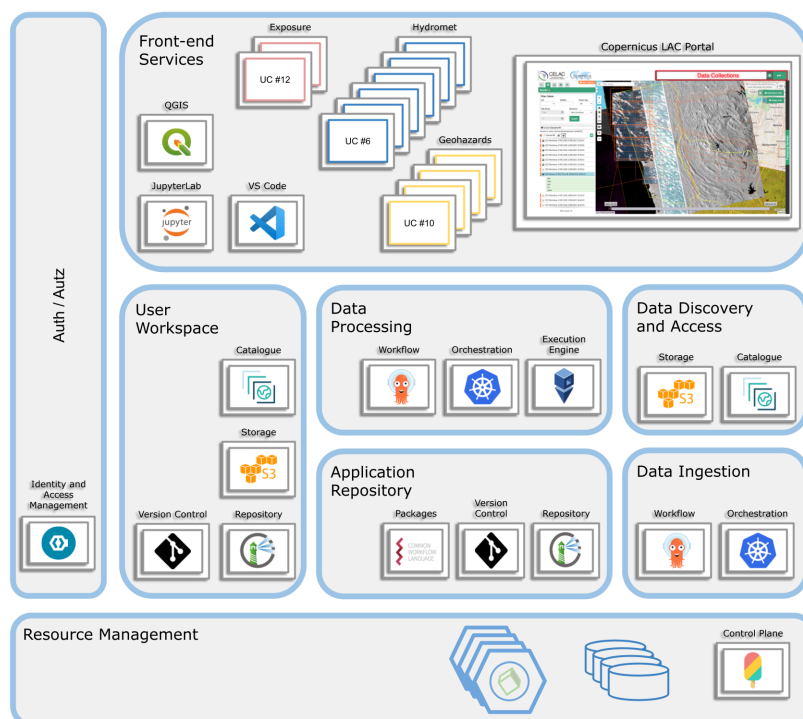


Figure 1 - CopernicusLAC Centre Platform System Components

2.2 Key Components

The platform comprises several core components, each of which plays a role in the system's overall functionality:

- **Data Ingestion**
 - **Functionality:** The Data Ingestion component is responsible for acquiring EO data from various sources, including satellites, ground stations, and external data providers. This component supports multiple data formats and ingestion protocols to accommodate a diverse range of data sources.
 - **Integration:** Data Ingestion interfaces with the Data Processing component to pass on validated and pre-processed data. It also interacts with the storage systems for cataloguing and long-term storage.
- **Data Processing**
 - **Functionality:** The Data Processing component is central to the platform's capability to transform raw EO data into actionable insights. It manages the execution of complex workflows, orchestrating tasks across a distributed computing environment.
 - **Integration:** This component interacts closely with Data Ingestion for processing incoming data and with Data Discovery and Access for storing and retrieving processed outputs. It also interfaces with the Application Execution Engine for deploying and running EO algorithms.
- **Data Discovery and Access**
 - **Functionality:** The Data Discovery and Access component allows users to search, retrieve, and visualise EO data. It provides tools for filtering datasets based on spatial, temporal, and thematic criteria, leveraging metadata cataloguing systems.
 - **Integration:** This component interfaces with the storage systems to access metadata and data files, and with Front-End Services to provide users with search and visualisation capabilities.
- **Front-End Services**
 - **Functionality:** Front-End Services provide the user interface for interacting with the CopernicusLAC platform. This includes the web portal, interactive development environments, and web applications that allow users to access data, run analyses, and visualise results.
 - **Integration:** These services interface with the Data Processing and Data Discovery components to deliver processed data and analysis tools to the end-users. They also interact with the User Workspace to provide a smooth user experience.
- **User Workspace**
 - **Functionality:** The User Workspace is a personalised environment where users can manage, process, and analyse EO data. It provides access to the necessary tools and resources in a private, user-specific setting.

- Integration: The User Workspace integrates with Front-End Services for data access and interaction, and with Data Processing for executing customised workflows.
- Security and Access Control
 - Functionality: This component ensures the platform's security by managing authentication, authorization, and access control. It safeguards data and services against unauthorised access and potential threats.
 - Integration: Security and Access Control interfaces with all other components to enforce security policies, manage user roles, and ensure compliance with regulatory standards. Keycloak is the central tool used for identity and access management across the platform.
- Application Packaging and Deployment
 - Functionality: The Application Packaging and Deployment component manages the lifecycle of EO applications, from packaging as Docker containers to deployment across the platform. It ensures that applications are consistent, scalable, and easily deployable.
 - Integration: This component interacts with the Data Processing and Resource Management components to deploy applications within the platform's infrastructure. It also interfaces with the Harbor container registry and Kubernetes for container management and orchestration.
- Resource Management
 - Functionality: Resource Management is responsible for provisioning, managing, and optimising the platform's computational and storage resources. This component ensures that the platform can scale to meet user demands.
 - Integration: Resource Management interfaces with Crossplane for dynamic resource provisioning and with monitoring tools to optimise resource usage across the platform. It plays a crucial role in maintaining the platform's performance and cost-effectiveness.

2.3 Component Interactions

The CopernicusLAC platform is designed with a focus on modularity and interoperability, ensuring that each component can operate independently while integrating with the others. The interaction between components is facilitated by a set of well-defined interfaces and protocols that manage data flow, communication, and execution of tasks across the platform, which are detailed in the sections of this document.

2.3.1 Data Flow from Ingestion to Processing

The data flow from ingestion to processing reflects a flexible and modular design that accommodates various levels of data ingestion, metadata registration, dataset copying, and dataset calibration. These levels influence how data moves through the system, ensuring

that ingestion workflows align with the dataset's intended use while maintaining efficiency and interoperability across components.

Data ingested through the Data Ingestion component follows a structured workflow. For metadata registration, only metadata is validated, cataloged, and indexed within the Data Discovery and Access component, enabling users to discover and retrieve datasets directly from their original sources. For dataset copying, the ingestion workflow includes the validation, storage, and indexing of datasets within the platform's S3-compatible object storage system. In scenarios involving dataset calibration, additional processing steps are executed during ingestion to prepare the data for specialized analyses or workflows. These calibrated datasets are also stored locally, accompanied by updated metadata reflecting the calibration processes.

The Data Discovery and Access component ensures that all ingested or registered data is indexed and made searchable using the STAC specification. This cataloging process provides a unified interface for users to explore datasets, regardless of their physical storage location.

When a user initiates a processing task, the Data Processing component retrieves the necessary datasets from the storage system or external access points defined during ingestion. The retrieval process is optimized to ensure that metadata alignment and dataset integrity are maintained. For locally stored datasets, the system uses Argo Workflows to orchestrate the execution of EO algorithms as defined in the Application Repository. These workflows manage dependencies and processing logic, ensuring that tasks such as data analysis, transformation, and visualization are performed.

For datasets registered via metadata only, the processing component retrieves data directly from the external source as needed, ensuring interoperability with distributed data systems. In cases involving calibrated datasets, the processing workflows leverage the pre-processed data for further analysis, reducing the computational overhead required during execution.

The platform's modularity ensures that the flow of data from ingestion to processing remains adaptable to varying user needs and operational requirements. Whether handling metadata-only ingestion for data discovery or managing full ingestion and calibration workflows for complex processing tasks, the system maintains integration across its components. This flexibility supports a wide range of use cases, from rapid metadata harvesting to large-scale EO data analysis.

2.3.2 Interaction between Data Processing and User Interfaces

As the Data Processing component completes the execution of workflows, the results are stored back into the storage system, with updated metadata added to the catalog. Users can then discover these processed datasets through the Data Discovery and Access interfaces.

Users interact with the platform via Front-end Services, which provide a unified interface to access data, run processing tasks, and visualise results. These services, including web applications, notebooks, and integrated development environments (IDEs), are tightly integrated with the User Workspace component, allowing users to manage their data and processing tasks.

2.3.3 Application Deployment and Execution

EO applications are packaged and stored in the Application Repository as Docker containers and associated Application Package manifest encoded using the Common Workflow Language (CWL). The containers include all necessary dependencies, ensuring that applications can be deployed consistently across different environments within the platform.

The deployment of these applications is managed by Kubernetes, which is responsible for orchestrating the containers across the platform's infrastructure. Helm charts are used to define deployment configurations, enabling repeatable and scalable deployment processes.

Once deployed, these applications can be executed by the Data Processing component or directly by users through the Front-end Services. The platform supports both batch processing and real-time processing, catering to a wide range of use cases.

2.3.4 Security and Access Control

The platform's security framework is centralised through the Authentication and Authorization component, which uses Keycloak for identity management. All access to data, applications, and services is controlled by Keycloak, which enforces policies based on user roles and permissions.

Multi-Factor Authentication (MFA) and Role-Based Access Control (RBAC) ensure that users only have access to the resources necessary for their roles, reducing the risk of unauthorised access. Security logs are maintained to track user activity, providing an audit trail for compliance and security analysis.

2.3.5 Resource Management and Scalability

The Resource Management component ensures that the platform can scale dynamically to meet user demand. The platform can automatically do the provisioning and deprovisioning of cloud resources based on workload and application requirements. This dynamic scaling capability ensures that the platform remains responsive even during periods of high demand.

The platform's architecture supports horizontal scaling, where additional instances of processing components can be deployed as needed. This approach allows the platform to handle large-scale EO data processing tasks without compromising performance.

3 Data Ingestion

Details the interfaces used for ingesting data from external sources, including protocols, data formats, and security measures.

3.1 Overview

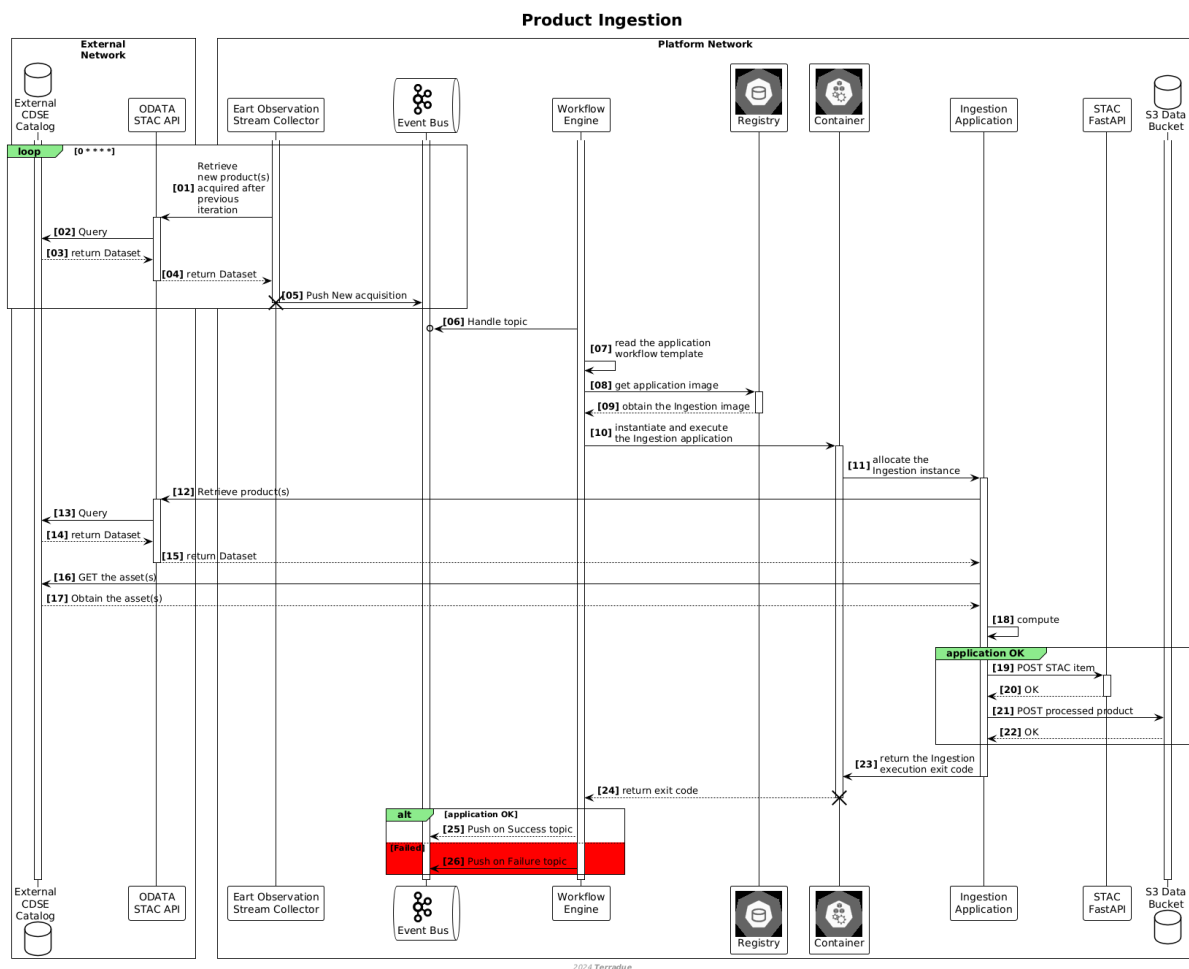


Figure 2 - Product Ingestion Sequence Diagram

The Data Ingestion interface in the CopernicusLAC system is responsible for managing the intake of EO data from external sources, particularly the Copernicus Data Space Ecosystem (CDSE). This process accommodates multiple levels of ingestion, depending on the dataset's role and user requirements:

- **Metadata Harvesting:** The platform registers metadata and access points for externally hosted datasets, enabling discovery and retrieval without duplicating the original data.

- Metadata Harvesting and Dataset Copying: In cases where datasets are essential for local processing or frequent use, both metadata and the datasets themselves are ingested and stored in the platform's infrastructure.
- Metadata Harvesting with Calibration: Specific datasets may undergo additional calibration or transformation workflows during ingestion to meet predefined analytical or quality requirements.

At its maximum extent, this process involves triggering the ingestion workflow, validating the data, and making it accessible within the platform for further processing and analysis.

The sequence diagram above outlines the flow of interactions between different components during the ingestion process.

3.2 Interface Requirements

3.2.1 Data Formats

The ingestion process expects a STAC Item describing the data which is the standard input for the ETL (Extract, Transform, Load) processes. The STAC Items assets should be in formats such as GeoTIFF, Cloud-Optimized GeoTIFF (COG), or other user-defined standards,

3.2.2 Initial Data Validation

Upon receiving the data, the system triggers the ingestion process via the Event Bus, which handles the topic related to new data acquisition. The ingestion workflow then reads the application workflow template from the Registry and obtains the necessary ingestion images.

3.2.3 Ingestion Process Execution

The ingestion instance is instantiated and executed by the Workflow Engine, which handles the data acquisition and processing. At its full scope the system computes the data, enriches and posts the STAC item to the STAC FastAPI, and stores the processed product in the S3 Data Bucket, however depending on the ingestion level the actions are:

- For metadata harvesting, the ingestion instance processes and indexes only the metadata, ensuring its alignment with the SpatioTemporal Asset Catalog (STAC) specification.
- For metadata harvesting and dataset copying, the ingestion workflow downloads the dataset, validates its integrity, and stores it in the platform's S3-compatible object storage.
- For metadata harvesting with calibration, the workflow applies predefined calibration routines, such as georeferencing or format conversion, before registering the dataset and its metadata for discovery and access.

3.2.4 Application Exit Codes

Upon successful execution, the system returns an exit code indicating the success of the ingestion process. If the application runs successfully, a message is pushed onto the success topic in the Event Bus. In the case of failure, the system pushes a message onto the failure topic.

3.3 Data Formats and Protocols

3.3.1 Accepted Data Format

The ingestion process supports multiple formats tailored to each ingestion level. For metadata harvesting, the system processes STAC-compliant metadata files. For dataset copying and calibration, the ingestion interface accepts data in formats such as GeoTIFF, Cloud-Optimized GeoTIFF (COG), or other user-defined standards, ensuring compatibility with downstream workflows.

3.3.2 STAC Specification

The STAC Item within the archive must include all mandatory fields, such as spatial and temporal metadata, and valid asset references.

3.3.3 Data Transfer Protocols

The data transfer protocol is not fully defined yet but is expected to involve secure methods such as HTTPS that is the data transfer protocol employed for the S3-based storage system. The final choice will be determined based on the capabilities of the services provided by the Copernicus Data Space Ecosystem.

3.4 Security and Compliance

3.4.1 Data Integrity

Data integrity is ensured through checksum verification during ingestion. The system will validate the integrity of the data as it is processed by the ingestion application.

3.4.2 Access Control

Access to the ingestion interface is restricted to authorised entities. The system uses secure authentication methods, potentially including OAuth2, to manage access control.

3.4.3 Compliance

The system is designed to comply with data protection regulations, ensuring that any sensitive data ingested is handled securely in accordance with GDPR and other relevant regulations.

3.5 Error Handling and Logging

3.5.1 Error Detection

The system continuously monitors the ingestion process for errors, considering the different levels:

- For metadata harvesting, it verifies metadata integrity and completeness.
- For dataset copying, it validates file integrity using checksum verification.
- For calibration workflows, it ensures that all transformation routines are executed correctly and flags any discrepancies for operator review.

If any issues arise, such as incorrect file formats or invalid STAC Items, the system will move the problematic data to temporary storage and notify the monitoring system.

3.5.2 Logging

Comprehensive logs are maintained throughout the ingestion process, capturing details of successful ingestions as well as errors. These logs are essential for troubleshooting and auditing purposes.

3.6 Future Work

This subsection provided the detailed process of data ingestion, highlighting the interactions between external repositories, the Event Bus, the Workflow Engine, and the various components involved in processing and storing the data. This diagram serves as a foundational reference for understanding the flow of data and control messages during ingestion.

Future iterations of the Data Ingestion interface will enhance support for more complex workflows, including user-defined calibration routines and dynamic prioritization of datasets based on usage patterns. These updates aim to further align ingestion processes with evolving user needs and platform objectives.

4 Data Discovery

4.1 Overview

The Data Discovery functionality in the CopernicusLAC system is provided through a catalog that utilises the SpatioTemporal Asset Catalog (STAC) REST API. This API enables the discovery and access of Earth Observation (EO) products by organising them into collections and allowing users to search and filter these collections based on specific criteria.

The STAC API is a widely adopted standard in the geospatial community, facilitating interoperability and ease of integration with other systems. This section will describe how data can be discovered using the STAC API, including methods for obtaining collection information, accessing metadata, filtering data, and retrieving specific items. Examples of API requests and responses are provided to illustrate the usage of these interfaces.

4.2 Interface Description

4.2.1 Obtain Collection Information

The STAC API allows users to obtain information about the available collections within the catalog. Two primary methods are available:

- GET /collections: Retrieves a JSON document containing all available collections.
- GET /collections/<collection-id>: Retrieves detailed information about a specific collection identified by <collection-id>.

Each collection is described by metadata, including spatial and temporal extents, links to further information, and other relevant details. Below is an example response for a collection:

```
{
  "id": "sentinel-s2-l1c-cogs",
  "type": "Collection",
  "links": [
    {
      "rel": "items",
      "type": "application/geo+json",
      "href": "<discovery-url>/collections/sentinel-s2-l1c-cogs/items"
    },
    {
      "rel": "parent",
      "type": "application/json",
      "href": "<discovery-url>/"
    },
    {
      "rel": "root",
      "type": "application/json",
      "href": "https://<discover-url>/"
    }
  ]
}
```

```

    },
    {
      "rel": "self",
      "type": "application/json",
      "href": "https://<discovery-url>/collections/sentinel-s2-l1c-cogs"
    },
    {
      "rel": "http://www.opengis.net/def/rel/ogc/1.0/queryables",
      "href": "<discovery-url>/collections/sentinel-s2-l1c-cogs/queryables",
      "type": "application/schema+json"
    }
  ],
  "title": "Sentinel Level-1C COGs",
  "extent": {
    "spatial": {
      "bbox": [ [ -180,-90,180,90 ] ]
    },
    "temporal": {
      "interval": [ [ "2023-06-01T00:00:00Z", "2023-06-30T23:59:59Z" ] ]
    }
  },
  "license": "proprietary",
  "keywords": [
    "copernicus",
    "esa",
    "eu",
    "msi",
    "radiance",
    "sentinel"
  ],
  "description": "Sentinel-2a and Sentinel-2b imagery, processed to Level 1C (TOA Reflectance) and converted to Cloud-Optimized ",
  "stac_version": "1.0.0"
}

```

4.2.2 Direct Metadata Access

Metadata about specific items within a collection can be accessed directly without filtering.

The following methods are supported:

- GET /collections/<collection-id>/items: Retrieves a GeoJSON FeatureCollection representing all items within the specified collection.
- GET /collections/<collection-id>/items/<item-id>: Retrieves a single GeoJSON Feature representing the item identified by <item-id>.

For example, to access metadata for a specific Sentinel-2 item, the following request can be made:

https://<discover-server>/collections/sentinel-s2-l1c-cogs/items/S2A_MSIL1C_20231122T102331_N0509_R065_T32TMR_20231122T122209

This request would return a GeoJSON Feature with detailed metadata about the specific Sentinel-2 product.

4.2.3 Obtain Filter Criteria

The STAC API provides mechanisms to obtain and utilise filter criteria, known as "queryables." These criteria allow users to refine their searches based on specific metadata fields such as date/time, geographic coverage, and cloud cover percentage. The filter criteria can be retrieved using the following method:

- GET /collections/<collection-id>/queryables:
Returns a JSON document with information about applicable filter criteria.

Example of a filter criteria response:

```
{
  ...
  "properties": {
    ...
    "datetime": {
      "type": "string",
      "title": "Acquired",
      "format": "date-time",
      "pattern": "(\\+00:00|Z)$",
      "description": "Datetime"
    },
    "geometry": {
      "$ref": "https://geojson.org/schema/Feature.json",
      "title": "Item Geometry",
      "description": "Item Geometry"
    },
    "eo:cloud_cover": {
      "$ref":
"https://stac-extensions.github.io/eo/v1.0.0/schema.json#/definitions/fieldsproperties/eo:cloud_cover"
    }
  }
}
```

This response provides the necessary information to construct search queries using the identified criteria.

4.3 Filtering

The STAC search endpoint supports two methods of filtering, allowing users to search for items that match specific criteria:

- GET /search: Uses URL query strings with filtering parameters (CQL2-TEXT).
- POST /search: Uses a request body containing filtering criteria formatted as a CQL2-JSON object.

Both methods achieve similar results, though HTTP POST allows for more complex queries. Below are examples of filtering using these methods.

4.3.1 Filtering using CQL2-JSON (POST)

Example of filtering by date/time:

```
{
  "collections": [
    "sentinel-s2-l1c-cogs"
  ],
  "filter-lang": "cql2-json",
  "filter": {
    "op": "T_INTERSECTS",
    "args": [
      {
        "property": "datetime"
      },
      {
        "interval": [
          "2023-11-22T00:00:00Z",
          "2023-11-23T00:00:00Z"
        ]
      }
    ]
  }
}
```

Example of filtering by geographic area:

```
{
  "collections": [
    "sentinel-s2-l1c-cogs"
  ],
  "filter-lang": "cql2-json",
  "filter": {
    "op": "S_INTERSECTS",
    "args": [
      {
        "property": "geometry"
      },
      {
        "type": "Polygon",
        "coordinates": [
          [
            [7.8, 45.2],
            [9.0, 45.2],
            [9.0, 45.8],
            [7.8, 45.8],
            [7.8, 45.2]
          ]
        ]
      }
    ]
  }
}
```

```
}  
]  
}  
}
```

Example of filtering by cloud cover percentage:

```
{  
  "collections": [  
    "sentinel-s2-l1c-cogs"  
  ],  
  "filter-lang": "cql2-json",  
  "filter": {  
    "op": "<",  
    "args": [  
      {  
        "property": "eo:cloud_cover"  
      },  
      10  
    ]  
  }  
}
```

Example of filtering by a combination of criteria (date/time and cloud cover percentage):

```
{  
  "collections": [  
    "sentinel-s2-l1c-cogs"  
  ],  
  "filter-lang": "cql2-json",  
  "filter": {  
    "op": "and",  
    "args": [  
      {  
        "op": "T_INTERSECTS",  
        "args": [  
          {  
            "property": "datetime"  
          },  
          {  
            "interval": [  
              "2023-11-22T00:00:00Z",  
              "2023-11-23T00:00:00Z"  
            ]  
          }  
        ]  
      }  
    ],  
  },  
  {  
    "op": "<",  
    "args": [  
      {  
        "property": "eo:cloud_cover"  
      },  
      10  
    ]  
  }  
}
```

```

        "property": "eo:cloud_cover"
      },
      0
    ]
  }
}
}
}

```

4.3.2 Filtering using CQL2-TEXT (GET)

This method provides a serialised form of querying similar to CQL2-JSON. The following are equivalent query strings for the examples above:

- Filter by date/time:
 - `filter-lang=cql2-text&collections=sentinel-s2-l1c-cogs&filter=datetime%3E=%272023-11-22T00:00:00Z%27%20and%20datetime%3C%272023-11-23T00:00:00Z%27`
- Filter by geographic area:
 - `filter-lang=cql2-text&collections=sentinel-s2-l1c-cogs&filter=S_INTERSECTS(geometry,%20POLYGON((7.8%2045.2,9.0%2045.2,9.0%2045.8,7.8%2045.8,7.8%2045.2)))`
- Filter by cloud cover percentage:
 - `filter-lang=cql2-text&collections=sentinel-s2-l1c-cogs&filter=eo:cloud_cover%3C10`
- Filter by a combination of criteria (date/time and cloud cover percentage):
 - `filter-lang=cql2-text&collections=sentinel-s2-l1c-cogs&filter=datetime%3E=%272023-11-22T00:00:00Z%27%20and%20datetime%3C%272023-11-23T00:00:00Z%27%20and%20eo:cloud_cover%3C10`

4.4 STAC Items

STAC Items are individual records within a collection, formatted as GeoJSON features. They represent specific EO products and include detailed metadata. Below is an example of a STAC Item for a Sentinel-2 product:

```

{
  "id": "S2A_MSIL1C_20231122T102331_N0509_R065_T32TMR_20231122T122209",
  "type": "Feature",
  "collection": "sentinel-s2-l1c-cogs",
  "links": [
    {
      "rel": "collection",
      "type": "application/json",
      "href": "<discovery-url>/collections/sentinel-s2-l1c-cogs"
    }
  ]
}

```

```

    },
    {
      "rel": "parent",
      "type": "application/json",
      "href": "<discovery-url>/collections/sentinel-s2-l1c-cogs"
    },
    {
      "rel": "root",
      "type": "application/json",
      "href": "<discovery-url>/\"
    },
    {
      "rel": "self",
      "type": "application/geo+json",
      "href":
"<discovery-url>/collections/sentinel-s2-l1c-cogs/items/S2A_MSIL1C_20231122T102331_N0509_R065_
T32TMR_20231122T122209\"
    },
    {
      "rel": "xyz",
      "href":
"<view-url>/collections/sentinel-s2-l1c-cogs/items/S2A_MSIL1C_20231122T102331_N0509_R065_T32TM
R_20231122T122209/tiles/WebMercatorQuad/{z}/{x}/{y}@1x?&assets=red&assets=green&assets=blue&co
lor_formula=Gamma%20RGB%201.5%20Saturation%201%20Sigmoidal%20RGB%2010%200.3&rescale=1,6000&res
cale=1,6000&rescale=1,6000\",
      "type": "image/png",
      "title": "True colour composite visualized through a XYZ\"
    },
    {
      "rel": "xyz",
      "href":
"<view-url>/collections/sentinel-s2-l1c-cogs/items/S2A_MSIL1C_20231122T102331_N0509_R065_T32TM
R_20231122T122209/tiles/WebMercatorQuad/{z}/{x}/{y}@1x?&assets=nir&assets=green&assets=red&col
or_formula=Gamma%20RGB%201.5%20Saturation%201%20Sigmoidal%20RGB%2010%200.3&rescale=1,6000&resc
ale=1,6000&rescale=1,6000\",
      "type": "image/png",
      "title": "False colour composite visualized through a XYZ\"
    }
  ],
  "assets": {
    "nir": {
      "href":
"s3://<storage-base>/sentinel-2-l1c-ingestion-xvv8c/S2A_MSIL1C_20231122T102331_N0509_R065_T32T
MR_20231122T122209/r-nir.tif\",
      "type": "image/tiff; application=geotiff; profile=cloud-optimized\",
      "roles": [
        \"data\",
        \"visual\",
        \"reflectance\"
      ],
      \"eo:bands\": [

```

```
{
  "name": "nir",
  "common_name": "nir",
  "description": "nir 833nm TOA 10",
  "center_wavelength": 0.8328,
  "solar_illumination": 1041.63
}
],
"raster:bands": [
  {
    "histogram": {
      "max": 27054.9431372549,
      "min": -51.94313725490196,
      "count": 256,
      "buckets": [

    ]
    },
    "statistics": {
      "mean": 1908.0699699159,
      "stddev": 1450.1018905764,
      "maximum": 27002.0,
      "minimum": 1.0,
      "valid_percent": 100.0
    },
    "spatial_resolution": 10.0
  }
],
"auth:refs": [
  "signed_url_auth"
]
},
...
},
"geometry": {
  "type": "Polygon",
  "coordinates": [
    [
      [ 7.706951084723532, 46.0462596089832 ],
      [ 7.706951084723532, 46.0462596089832 ],
      [ 7.729416423067862, 45.058191670236305 ],
      [ 9.123974498511316, 45.06519270654337 ],
      [ 9.126167237224049, 46.05350473573164 ],
      [ 7.706951084723532, 46.0462596089832 ]
    ]
  ]
},
"bbox": [
  7.706951084723532, 45.058191670236305,
  9.126167237224049, 46.05350473573164
],
```

```

"properties": {
  "gsd": 10.0,
  "title": "SENTINEL-2A MSI L1C 65 2023-11-22 10:23:31",
  "created": "2023-11-22T12:22:09Z",
  "mission": "sentinel-2",
  "updated": "2024-01-23T11:41:41.147340Z",
  "datetime": "2023-11-22T10:23:31.024000Z",
  "platform": "sentinel-2a",
  "proj:epsg": 32632,
  "providers": [
    {
      "url": "https://sentinel.esa.int/web/sentinel/missions/sentinel-2",
      "name": "ESA/EC (Copernicus)",
      "roles": [
        "producer",
        "processor",
        "licensor"
      ],
      "description": "The Copernicus Sentinel-2 mission comprises a constellation of two
polar-orbiting satellites placed in the same sun-synchronous orbit, phased at 180\u00b0 to
each other. It aims at monitoring variability in land surface conditions, and its wide swath
width (290 km) and high revisit time (10 days at the equator with one satellite, and 5 days
with 2 satellites under cloud-free conditions which results in 2-3 days at mid-latitudes) will
support monitoring of Earth's surface changes."
    }
  ],
  "description": "Name |Value\n---- |-----\nAgency | **ESA** \nProduct Id|
**S2A_MSIL1C_20231122T102331_N0509_R065_T32TMR_20231122T122209** \nPlatform | **sentinel-2a**
\nInstrument | **MSI** \nSource processing level | **L1C** \nSensing Time | **2023-11-22
10:23:31** \nService version | **NO_VERSION** \nUpdated | **2024-01-23 11:41:41** \nStatus |
**Calibrated Dataset** \nCreated | **2023-11-22 12:22:09** \n",
  "instruments": [
    "msi"
  ],
  "sensor_type": "optical",
  "constellation": "sentinel-2",
  "eo:cloud_cover": 0.243363492490071,
  "sat:orbit_state": "descending",
  "processing:level": "L1C",
  "processing:lineage": "Generation of Level-1C User Product",
  "sat:absolute_orbit": 43963,
  "sat:relative_orbit": 65,
  "processing_baseline": "05.09",
  "sat:platform_international_designator": "2015-000A"
  "auth:schemes": {
    "signed_url_auth": {
      "type": "signedUrl",
      "flows": {
        "authorizationCode": {
          "method": "POST",
          "parameters": {

```

```
        "url": {
          "in": "body",
          "schema": {
            "type": "string",
            "examples":
"s3://<storage-base>/optical-calibration-sj889/S2B_MSIL2A_20230922T095649_N0509_R122_T33TUG_20
230922T125953-calibrated/red.tif"
          },
          "required": "true",
          "description": "asset URL"
        },
        "redirect": {
          "in": "body",
          "schema": {
            "type": "boolean",
            "examples": "true"
          },
          "required": "true",
          "description": "indicate if the response include a 302 for an automatic
redirect with 'Location' header"
        },
        "Authorization": {
          "in": "header",
          "schema": {
            "type": "string",
            "examples": "Bearer secretKey1234567890"
          },
          "required": "true",
          "description": "Authentication credentials"
        }
      },
      "responseField": "signed_url",
      "authorizationApi": "<sign-url>"
    },
    "description": "Provides an authenticated download link for an asset URL"
  }
},
"stac_extensions": [
  "https://stac-extensions.github.io/sat/v1.0.0/schema.json",
  "https://stac-extensions.github.io/eo/v1.1.0/schema.json",
  "https://stac-extensions.github.io/raster/v1.1.0/schema.json",
  "https://stac-extensions.github.io/projection/v1.1.0/schema.json",
  "https://stac-extensions.github.io/processing/v1.0.0/schema.json",
  "https://stac-extensions.github.io/view/v1.0.0/schema.json",
  "https://stac-extensions.github.io/web-map-links/v1.2.0/schema.json",
  "https://stac-extensions.github.io/authentication/v1.0.0/schema.json"
],
"stac_version": "1.0.0"
}
```

Note the following information (highlighted in red above):

- Links point to related resources, such as the collection or the catalogue.
- Among the links, there are also visualisation links to a tile server (see Tiles).
- Assets are downloadable resources containing the original data represented by the STAC item (image files, data files, other metadata files etc.).
- An asset may have information about the authentication method to be used for the download (e.g. signed URLs). In that case, there is a reference to an authentication scheme defined at the item/properties level (see below)
- Properties represent basic metadata fields (collection-specific) that can be used for filtering.
- Among the properties there can be a series of authentication schemes, i.e. instructions to which assets can refer in their download authentication information (see above).

5 Data Visualisation

5.1 Overview

The Data Visualization component of the CopernicusLAC system provides users with the capability to visually explore EO data through various forms of visual representations. This section describes the methods and formats used to visualise data, including overview images and Web Map Tile Service (WMTS) endpoints, which are integral for creating interactive maps in web applications.

5.2 Visualisation Types

There are two primary types of visualisations supported by the CopernicusLAC system:

1. **Overview Images:** These are static images providing a quick visual summary of a product.
2. **WMTS Endpoints:** These provide tile-based visualisations that can be used in interactive maps, allowing users to explore data at different zoom levels and geographic extents.

5.3 Overview Images

Overview images are simplified visual representations of the EO products, designed to provide a quick preview of the data. These images are accessible via a specific STAC Item asset, which includes the following key attributes:

- **href:** The URL that points to the location of the overview image.
- **role:** Set to **"overview"**, indicating that this asset is intended as a preview of the product.

Example of a STAC Item with an overview image:

```
-
{
  ...
  features: [
    {
      ...
      "assets": {
        ...
        "overview": {
          "href":
"<view-url>/collections/sentinel-s2-l1c-cogs/items/S2A_MSIL1C_20231122T102331_N0509_R065_T32TM
R_20231122T122209/preview.png?&assets=red&assets=green&assets=blue&color_formula=Gamma%20RGB%2
01.5%20Saturation%201%20Sigmoidal%20RGB%2010%200.3&max_size=1024&resampling=cubic&rescale=1,75
00&rescale=1,7500&rescale=1,7500",
          "type": "image/png",

```

```
    "roles": [
      "overview"
    ],
    "title": "Overview"
  },
  ...
}
```

This example shows how the `href` attribute links to an image, which is generated with specific parameters, such as resampling method and colour formula, to create a visually effective overview of the EO data.

5.4 Web Map Tile Service (WMTS) Endpoints

The CopernicusLAC system provides WMTS endpoints for visualising EO data in web applications. These endpoints allow for the dynamic rendering of data as tiles, which can be integrated into interactive maps, enabling users to zoom in and out and explore the data geographically.

5.4.1 Tile Visualisations

Multiple tile visualisations may be available for a single product, such as true colour and false colour composites. These visualisations are accessed via tile link templates provided in the STAC Item's links section. These templates follow the Web Map Links STAC extension and are identified by their titles.

Example of a STAC Item with tile links:

```
{
  ...
  features: [
    {
      ...
      "links": [
        {
          "rel": "xyz",
          "href":
"<view-url>/collections/sentinel-s2-l1c-cogs/items/S2A_MSIL1C_20231122T102331_N0509_R065_T32TM
R_20231122T122209/tiles/WebMercatorQuad/{z}/{x}/{y}@1x?&assets=red&assets=green&assets=blue&co
lor_formula=Gamma%20RGB%201.5%20Saturation%201%20Sigmoidal%20RGB%2010%200.3&rescale=1,6000&res
cale=1,6000&rescale=1,6000",
          "type": "image/png",
          "title": "True colour composite visualised through a XYZ"
        },
        {
          "rel": "xyz",
```

```
    "href":  
    "<view-url>/collections/sentinel-s2-l1c-cogs/items/S2A_MSIL1C_20231122T102331_N0509_R065_T32TM  
R_20231122T122209/tiles/WebMercatorQuad/{z}/{x}/{y}@1x?&assets=nir&assets=green&assets=red&col  
or_formula=Gamma%20RGB%201.5%20Saturation%201%20Sigmoidal%20RGB%2010%200.3&rescale=1,6000&resc  
ale=1,6000&rescale=1,6000",  
    "type": "image/png",  
    "title": "False colour composite visualised through a XYZ"  
  }  
  ...  
}  
],  
...  
]  
}
```

In this example:

- The `rel` attribute is set to "xyz", indicating that these are XYZ tile links, a common format for web mapping applications.
- The `href` attribute includes placeholders {z}, {x}, and {y}, which must be replaced with the zoom level and the zero-based column and row numbers to fetch the appropriate tile image.

5.4.2 Tile Request Format

To request a specific tile, the placeholders {z}, {x}, and {y}, in the tile link template must be replaced with actual values corresponding to the desired zoom level and geographic location on the tile grid. For example:

```
<view-url>/collections/sentinel-s2-l1c-cogs/items/S2A_MSIL1C_2023112  
2T102331_N0509_R065_T32TMR_20231122T122209/tiles/WebMercatorQuad/10/  
525/391@1x?&assets=red&assets=green&assets=blue&color_formula=Gamma%  
20RGB%201.5%20Saturation%201%20Sigmoidal%20RGB%2010%200.3&rescale=1,  
6000&rescale=1,6000&rescale=1,6000
```

In this request:

- The {z} placeholder is replaced with 10, indicating the zoom level.
- The {x} and {y} placeholders are replaced with 525 and 391, representing the tile's column and row in the grid, respectively.

This URL will return a tile image showing the true colour composite of the EO product at the specified zoom level and location.

6 Data Access Interfaces

6.1 Overview

The Data Access interfaces in the CopernicusLAC system are designed to facilitate secure and efficient access to EO data stored in object storage. These interfaces allow users to download specific assets from the system, with access control enforced via pre-signed URLs. This section provides an overview of the mechanisms used to generate and use these signed URLs, ensuring that only authorised users can retrieve the data.

The signed URLs interface is designed primarily for machine-to-machine (M2M) access to S3-compatible object storage resources. Upon authentication and authorization, a client may request a pre-signed HTTPS URL that grants time-limited access to a specific object, without requiring the client to handle or store long-term credentials. This mechanism ensures secure, auditable, and scalable access to EO datasets and platform-generated products.

This interface is also used internally by the platform components and externally by authenticated clients. More specifically:

- Web UI Integration (Human-to-Machine access): The CopernicusLAC geobrowser interface will use this API transparently to allow users to download data via a one-click interaction. The front-end will handle the retrieval of a signed URL and trigger the download automatically, ensuring ease of use without exposing access tokens or object paths directly.
- Workflow Integration (Machine-to-Machine access): For data processing services integrated via the CWL, there is no need for services to generate or manage signed URLs themselves. Instead, the platform handles data staging through a stage-in mechanism. This process ensures that input datasets are made available as local file paths inside the containerized service execution environment, avoiding the need for remote access logic within the service components.

This approach ensures consistency with best practices for data access security, scalability for high-throughput processing, and usability for human-facing interfaces.

6.2 Downloading Data with Signed URLs

Accessing STAC item assets that are stored in object storage requires generating a pre-signed URL. This URL grants temporary access to the asset, allowing it to be downloaded via HTTP. The pre-signing process is handled through a specific authentication scheme, typically using the **signed_url_auth** method as defined in the STAC item metadata.

6.2.1 Authentication Scheme

The **signed_url_auth** scheme is a standard approach for securing access to assets in object storage. The scheme is defined within the STAC item's properties and specifies how a pre-signed URL can be obtained.

Example of a STAC item asset with the **signed_url_auth** scheme:

```
{
  ...
  features: [
    {
      ...
      "properties": {
        ...
        "auth:schemes": {
          "signed_url_auth": {
            "type": "signedUrl",
            "flows": {
              "authorizationCode": {
                "method": "POST",
                "parameters": {
                  "url": {
                    "in": "body",
                    "schema": {
                      "type": "string",
                      "examples":
"s3://<storage-base>/optical-calibration-sj889/S2B_MSIL2A_20230922T095649_N0509_R122_T33TUG_20
230922T125953-calibrated/red.tif"
                    },
                    "required": "true",
                    "description": "asset URL"
                  },
                  "redirect": {
                    "in": "body",
                    "schema": {
                      "type": "boolean",
                      "examples": "true"
                    },
                    "required": "true",
                    "description": "indicate if the response include a 302 for an automatic
redirect with 'Location' header"
                  },
                  "Authorization": {
                    "in": "header",
                    "schema": {
                      "type": "string",
                      "examples": "Bearer secretKey1234567890"
                    },
                    "required": "true",
                    "description": "Authentication credentials"
                  }
                }
              }
            }
          }
        }
      }
    }
  ]
}
```

```

    }
    },
    "responseField": "signed_url",
    "authorizationApi": "https://example.com/signed_url/authorize"
  }
  },
  "description": "Provides an authenticated download link for an asset URL"
}
},
...
}
]
}
}

```

In this example, the asset located at the specified S3 URL is protected by the **signed_url_auth** scheme, meaning that a signed URL must be generated to access it.

The authentication scheme to be used is specified in the asset information of the STAC item, e.g.:

```

{
  ...
  "assets": {
    "nir": {
      "href":
"s3://<storage-base>/sentinel-2-l1c-ingestion-xvv8c/S2A_MSIL1C_20231122T102331_N0509_R065_T32T
MR_20231122T122209/r-nir.tif",
      ...
      "auth:refs": [
        "signed_url_auth"
      ],
      ...
    },
    ...
  },
  ...
}

```

6.2.2 Generating a Signed URL

To generate a signed URL, a POST request must be made to an authorization endpoint. This request includes the S3 URL of the asset as well as authentication credentials.

Example request using cURL:

```

curl -X 'POST' \
  'https://<sign-url>?url=s3%3A%2F%2Fsentinel-2%2Fred.tif' \
  -H 'Authorization: Bearer eyJ...cmQ' \

```

-d ''

In this request:

- The url parameter contains the encoded S3 URL of the asset.
- The Authorization header includes a Bearer token, which is used to authenticate the request.

6.2.3 Response and Redirection

The response to the POST request is a JSON object that contains the signed URL, which can be used to download the asset. If the redirect parameter is set to true, the response will also include a Location header that automatically redirects the user to the signed URL.

Example response:

```
{
  "signed_url":
"<storage-url>/sentinel-2/red.tif?X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=36TH83i1lW
k0seYhN96Z%2F20240122%2Fit-rom%2Fs3%2Faws4_request&X-Amz-Date=20240122T134237Z&X-Amz-Expires=1
20&X-Amz-SignedHeaders=host&X-Amz-Signature=2fc41c51180a954d93f38b2b868ad9026328fa384a975305d9
052715fdbef80b"
}
```

If redirection is enabled, the resource will be automatically downloaded from the signed URL provided in the response.

6.2.4 Example Workflow

1. Request Signed URL: The user or application requests a signed URL by sending a POST request to the authorization endpoint with the asset's S3 URL and authentication credentials.
2. Receive Signed URL: The server responds with a signed URL that provides temporary access to the asset.
3. Download Asset: The user or application can then use the signed URL to download the asset directly from the object storage.

6.3 Practical Example

Consider a scenario where a user wants to download a specific asset from the CopernicusLAC system, such as a NIR band image from Sentinel-2 data. The process would be as follows:

1. Identify the Asset

The user identifies the desired asset from the STAC item metadata, noting the S3 URL and the required authentication scheme (signed_url_auth).

2. Generate Signed URL:

The user sends a POST request to the authorization endpoint with the asset's S3 URL. Example cURL request:

```
curl -X 'POST' \  
  'https://<sign-url>?url=s3%3A%2F%2Fsentinel-2%2Fred.tif' \  
  -H 'Authorization: Bearer eyJ...cmQ' \  
  -d ''
```

3. Download the Asset:

Upon receiving the signed URL in the response, the user can either manually download the asset using the provided URL or rely on the automatic redirection (if enabled) to initiate the download immediately.

This method ensures that data access within the CopernicusLAC system is both secure and efficient, protecting sensitive EO data while providing users with the flexibility to download assets as needed.